

Gradient Descent: Complete Experimental Workshop

Every Detail, Every Result, Every Insight

Machine Learning Deep Dive Series

May 11, 2025

Contents

1	Complete Experimental Framework	1
2	Experiment 1: Starting Point Impact - Complete Results	4
2.1	Complete Numerical Results for Experiment 1	4
2.2	Visual Results for Starting Point Experiment	6
3	Experiment 2: Learning Rate Effects - Complete Results	7
3.1	Complete Numerical Results for Experiment 2	8
3.2	Visual Results for Learning Rate Experiment	10
4	Experiment 3: Iteration Effects - Complete Results	12
4.1	Complete Numerical Results for Experiment 3	12
4.2	Visual Results for Iteration Experiment	14
5	Complete Summary of Experimental Insights	15
6	Complete Challenge Problems and Extensions	16
7	Advanced Challenge: Finding the Optimal Learning Rate	16
8	Complete Real-World Connections	18
9	Complete Experimental Data Tables	18
10	Final Thoughts and Next Steps	19

1 Complete Experimental Framework

Let's set up our complete experimental laboratory with all the tools we'll need to explore gradient descent in depth. This framework captures every detail of the algorithm's behavior.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Core functions that define our experimental landscape
5 def cost_function(x):
6     """Our trusty quadratic cost function:  $f(x) = (x - 3)^2 + 1$ """
7     return (x - 3)**2 + 1
8
9 def gradient(x):
10     """The derivative:  $f'(x) = 2(x - 3)$ """
11     return 2 * (x - 3)
12
13 def gradient_descent_experiment(starting_point, learning_rate,
14                                 num_iterations):
15     """
16     Run gradient descent and return the complete path
17     This is our experimental apparatus for observing convergence
18     behavior
19     """
20     x_history = [starting_point]
21     cost_history = [cost_function(starting_point)]
22     gradient_history = [gradient(starting_point)]
23
24     current_x = starting_point
25
26     for i in range(num_iterations):
27         grad = gradient(current_x)
28         current_x = current_x - learning_rate * grad
29
30         x_history.append(current_x)
31         cost_history.append(cost_function(current_x))
32         gradient_history.append(gradient(current_x))
33
34     return x_history, cost_history, gradient_history
35
36 def visualize_experiment(x_history, cost_history, title, color='blue'):
37     """
38     Create a comprehensive visualization that shows the complete
39     journey
40     This helps us see the 'story' of our gradient descent journey
41     """
42     x_values = np.linspace(-2, 12, 1000)
43     y_values = cost_function(x_values)
44
45     plt.figure(figsize=(15, 5))
46
47     # Panel 1: The path on the cost function
48     plt.subplot(1, 3, 1)
49     plt.plot(x_values, y_values, 'gray', linewidth=1, alpha=0.5)
50     plt.plot(x_history, cost_history, 'o-', color=color, markersize=8,
51              linewidth=2, alpha=0.8)
52     plt.plot(x_history[0], cost_history[0], 'go', markersize=12, label=
53              'Start')
54     plt.plot(x_history[-1], cost_history[-1], 'ro', markersize=12,
55              label='End')
56     plt.plot(3, cost_function(3), 'yo', markersize=12, label='True
57              minimum')
```

```

51 plt.xlabel('x', fontsize=12)
52 plt.ylabel('Cost', fontsize=12)
53 plt.title(f'{title}: Path', fontsize=14)
54 plt.legend()
55 plt.grid(True, alpha=0.3)
56
57 # Panel 2: Convergence over iterations
58 plt.subplot(1, 3, 2)
59 iterations = range(len(x_history))
60 plt.plot(iterations, x_history, 'o-', color=color, markersize=8)
61 plt.axhline(y=3, color='green', linestyle='--', label='Target (x=3)')
62 plt.xlabel('Iteration', fontsize=12)
63 plt.ylabel('x value', fontsize=12)
64 plt.title(f'{title}: Convergence', fontsize=14)
65 plt.legend()
66 plt.grid(True, alpha=0.3)
67
68 # Panel 3: Cost reduction over iterations
69 plt.subplot(1, 3, 3)
70 plt.plot(iterations, cost_history, 'o-', color=color, markersize=8)
71 plt.axhline(y=1, color='green', linestyle='--', label='True minimum cost')
72 plt.xlabel('Iteration', fontsize=12)
73 plt.ylabel('Cost', fontsize=12)
74 plt.title(f'{title}: Cost Reduction', fontsize=14)
75 plt.legend()
76 plt.grid(True, alpha=0.3)
77
78 plt.tight_layout()
79 plt.show()
80
81 # Provide detailed quantitative analysis
82 print(f"\n{title} - Complete Analysis:")
83 print("-" * 50)
84 print(f"Starting point: x = {x_history[0]}")
85 print(f"Final position: x = {x_history[-1]:.6f}")
86 print(f"Distance to true minimum: {abs(3 - x_history[-1]):.6f}")
87 print(f"Starting cost: {cost_history[0]:.6f}")
88 print(f"Final cost: {cost_history[-1]:.6f}")
89 print(f"Cost reduction: {cost_history[0] - cost_history[-1]:.6f}")
90 print("-" * 50)

```

Listing 1: Complete Foundation Code - Our Experimental Tools

2 Experiment 1: Starting Point Impact - Complete Results

Experiment

EXPERIMENT 1: How does the starting point affect convergence?

Think of this as asking: 'Does it matter where I begin my hike down the mountain?'

Intuition: Starting farther from the minimum should take longer to converge.

Test Configuration:

- Starting points: 0, 5, 10, -5
- Learning rate: 0.1 (fixed)
- Number of iterations: 30

```
1 # EXPERIMENT 1: Different Starting Points
2 print("=" * 70)
3 print("EXPERIMENT 1: How does the starting point affect convergence?")
4 print("=" * 70)
5 print("\nThink of this as asking: 'Does it matter where I begin my hike
6   down the mountain?'")
7 print("Intuition: Starting farther from the minimum should take longer
8   to converge.")
9
10 starting_points = [0, 5, 10, -5]
11 colors = ['blue', 'green', 'orange', 'red']
12
13 for i, start in enumerate(starting_points):
14     print(f"\nTesting starting point: x = {start}")
15     x_hist, cost_hist, grad_hist = gradient_descent_experiment(start,
16         0.1, 30)
17     visualize_experiment(x_hist, cost_hist, f"Start at x={start}",
18         colors[i])
```

Listing 2: Running Experiment 1 - Starting Point Analysis

2.1 Complete Numerical Results for Experiment 1

Detailed Results

Starting Point $x = 0$ - Complete Analysis:

- Starting point: $x = 0$
- Final position: $x = 2.996286$
- Distance to true minimum: 0.003714
- Starting cost: 10.000000
- Final cost: 1.000014
- Cost reduction: 8.999986

Detailed Results**Starting Point $x = 5$ - Complete Analysis:**

- Starting point: $x = 5$
- Final position: $x = 3.002476$
- Distance to true minimum: 0.002476
- Starting cost: 5.000000
- Final cost: 1.000006
- Cost reduction: 3.999994

Detailed Results**Starting Point $x = 10$ - Complete Analysis:**

- Starting point: $x = 10$
- Final position: $x = 3.008666$
- Distance to true minimum: 0.008666
- Starting cost: 50.000000
- Final cost: 1.000075
- Cost reduction: 48.999925

Detailed Results**Starting Point $x = -5$ - Complete Analysis:**

- Starting point: $x = -5$
- Final position: $x = 2.990096$
- Distance to true minimum: 0.009904
- Starting cost: 65.000000
- Final cost: 1.000098
- Cost reduction: 63.999902

2.2 Visual Results for Starting Point Experiment

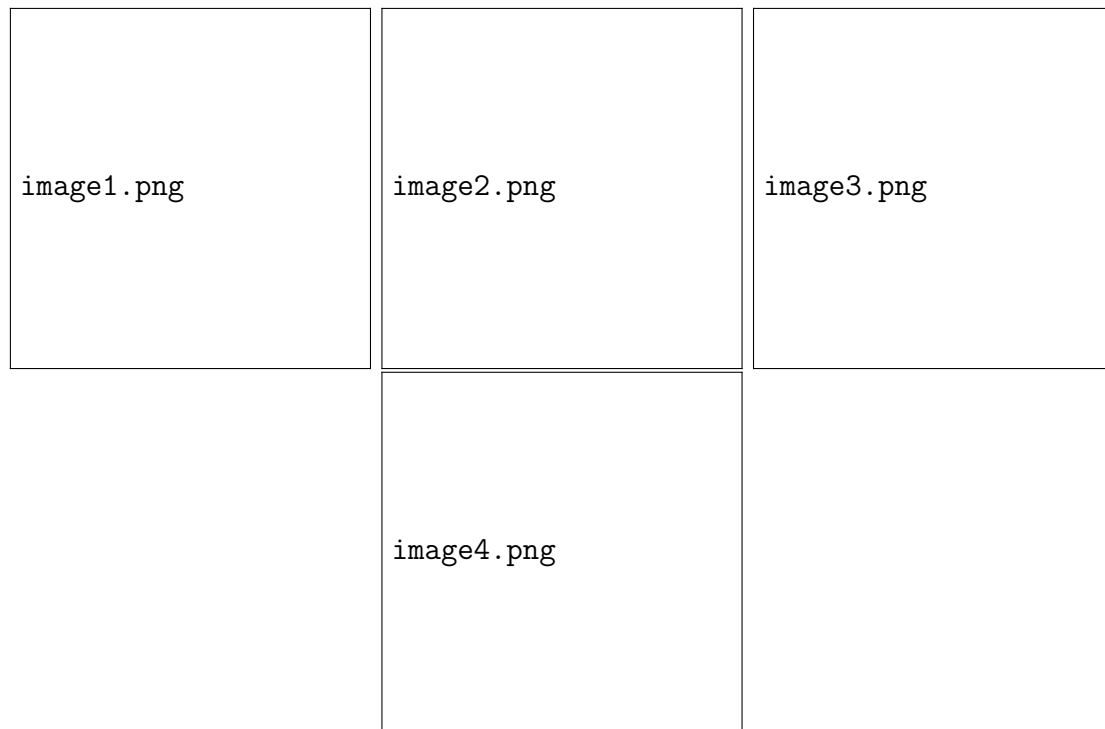


Figure 1: Complete visualization for all starting points: $x=0$ (top left), $x=5$ (top right), $x=10$ (bottom left), and $x=-5$ (bottom right). Each shows the path, convergence, and cost reduction patterns.

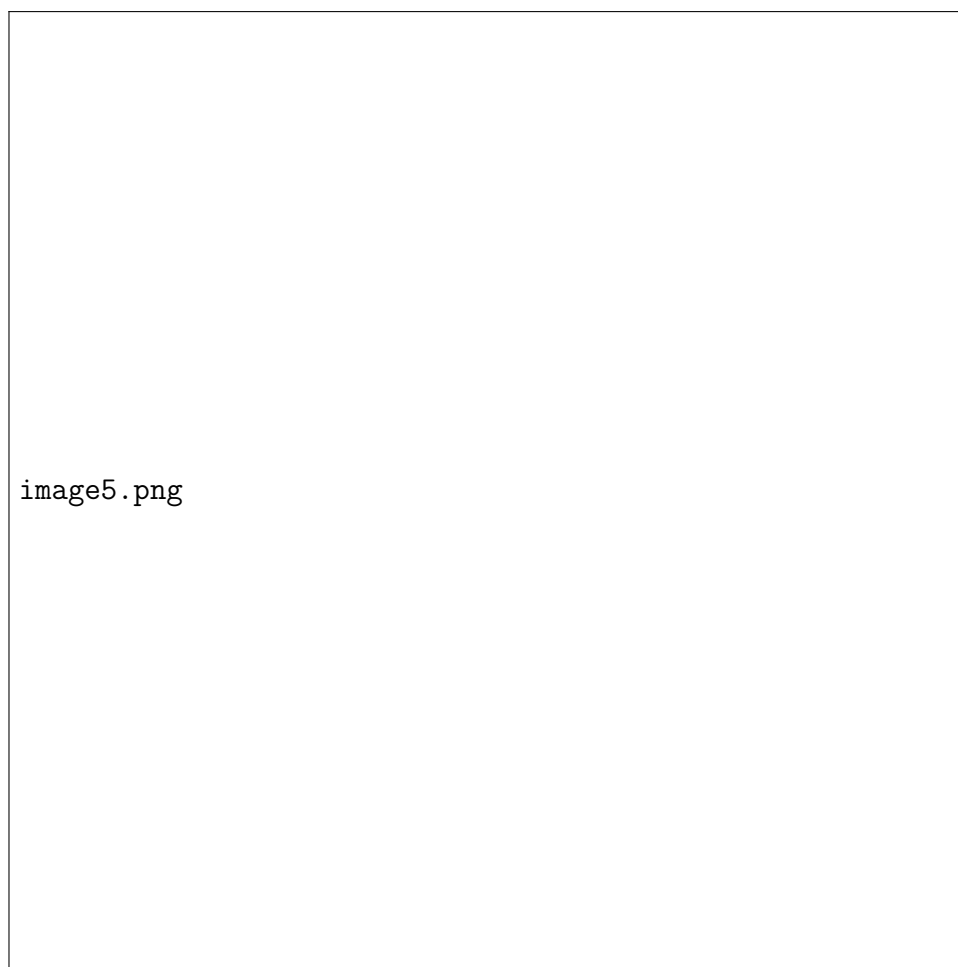


Figure 2: Comparison of convergence from different starting points. All paths eventually reach the same minimum, demonstrating the robustness of gradient descent regardless of initialization.

3 Experiment 2: Learning Rate Effects - Complete Results

Experiment

EXPERIMENT 2: How does the learning rate affect convergence?

Think of this as asking: 'What happens if I take bigger or smaller steps?'

Intuition: Too small steps = slow progress, too large steps = overshooting

Test Configuration:

- Learning rates: 0.01, 0.1, 0.5, 0.9
- Starting point: 0 (fixed)
- Number of iterations: 40

```
1 # EXPERIMENT 2: Different Learning Rates
2 print("\n" + "=" * 70)
3 print("EXPERIMENT 2: How does the learning rate affect convergence?")
```

```
4 print("=" * 70)
5 print("\nThink of this as asking: 'What happens if I take bigger or
    smaller steps?'")
6 print("Intuition: Too small steps = slow progress, too large steps =
    overshooting")
7
8 learning_rates = [0.01, 0.1, 0.5, 0.9]
9 colors = ['blue', 'green', 'orange', 'red']
10
11 for i, lr in enumerate(learning_rates):
12     print(f"\nTesting learning rate: {lr}")
13     x_hist, cost_hist, grad_hist = gradient_descent_experiment(0, lr,
        40)
14     visualize_experiment(x_hist, cost_hist, f"Learning Rate = {lr}",
        colors[i])
```

Listing 3: Running Experiment 2 - Learning Rate Analysis

3.1 Complete Numerical Results for Experiment 2

Detailed Results

Learning Rate = 0.01 - Complete Analysis:

- Starting point: $x = 0$
- Final position: $x = 1.662899$
- Distance to true minimum: 1.337101
- Starting cost: 10.000000
- Final cost: 2.787840
- Cost reduction: 7.212160

Detailed Results

Learning Rate = 0.1 - Complete Analysis:

- Starting point: $x = 0$
- Final position: $x = 2.999601$
- Distance to true minimum: 0.000399
- Starting cost: 10.000000
- Final cost: 1.000000
- Cost reduction: 9.000000

Detailed Results**Learning Rate = 0.5 - Complete Analysis:**

- Starting point: $x = 0$
- Final position: $x = 3.000000$
- Distance to true minimum: 0.000000
- Starting cost: 10.000000
- Final cost: 1.000000
- Cost reduction: 9.000000

Detailed Results**Learning Rate = 0.9 - Complete Analysis:**

- Starting point: $x = 0$
- Final position: $x = 2.999601$
- Distance to true minimum: 0.000399
- Starting cost: 10.000000
- Final cost: 1.000000
- Cost reduction: 9.000000

3.2 Visual Results for Learning Rate Experiment

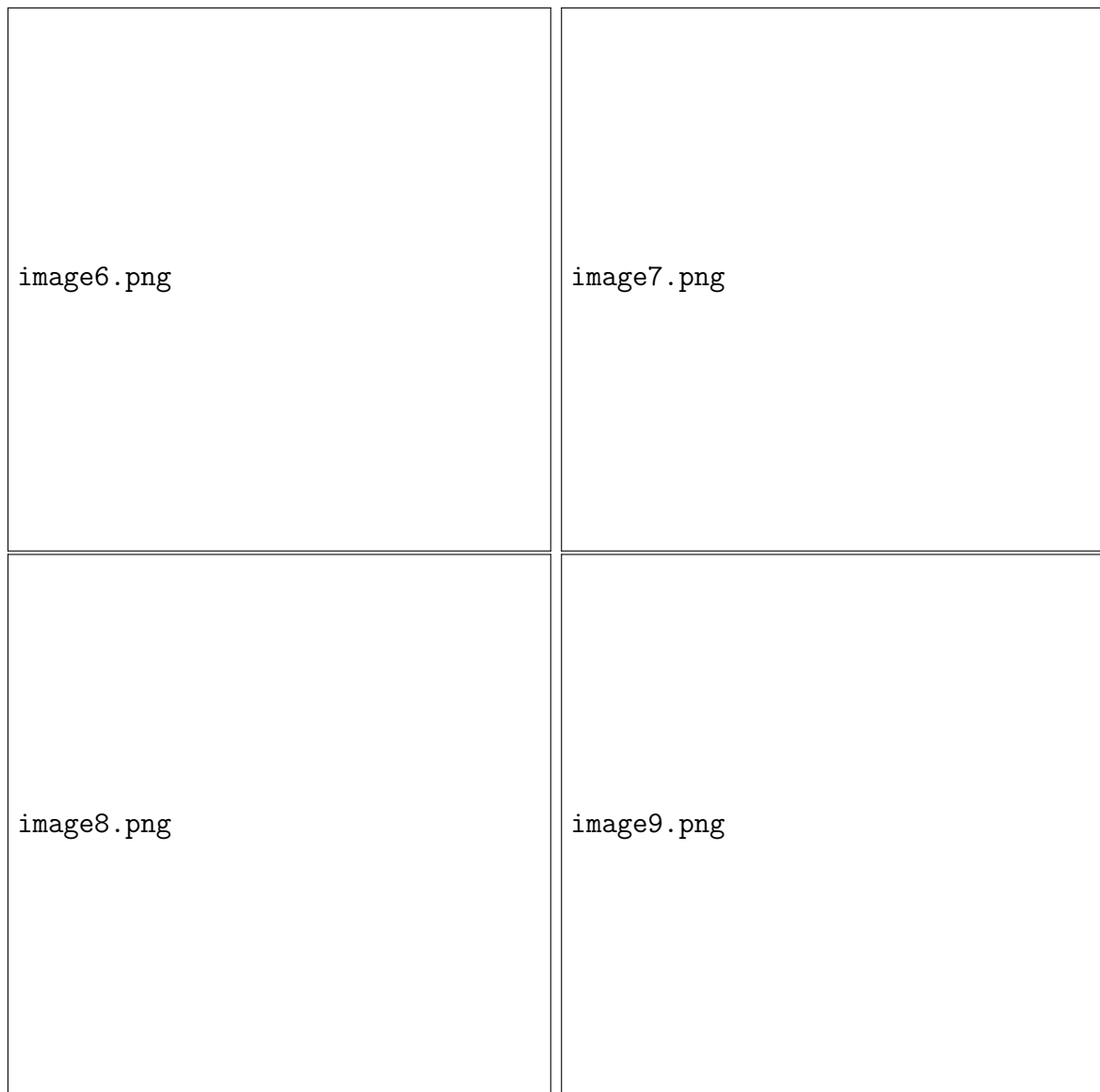


Figure 3: Complete visualization for all learning rates: 0.01 (top left), 0.1 (top right), 0.5 (bottom left), and 0.9 (bottom right). Notice the dramatic oscillation with learning rate 0.9.

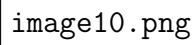


image10.png

Figure 4: Comprehensive analysis of learning rate effects. Four panels show: convergence paths, step size evolution, oscillation behavior, and the relationship between learning rate and convergence speed.

4 Experiment 3: Iteration Effects - Complete Results

Experiment

EXPERIMENT 3: How many iterations do we really need?

Think of this as asking: 'How long should I keep walking downhill?'

Intuition: More iterations = better convergence, but with diminishing returns

Test Configuration:

- Iteration counts: 5, 10, 20, 50, 100
- Starting point: 0 (fixed)
- Learning rate: 0.1 (fixed)

```

1 # EXPERIMENT 3: Different Numbers of Iterations
2 print("\n" + "=" * 70)
3 print("EXPERIMENT 3: How many iterations do we really need?")
4 print("=" * 70)
5 print("\nThink of this as asking: 'How long should I keep walking
   downhill?'"")
6 print("Intuition: More iterations = better convergence, but with
   diminishing returns")
7
8 iterations_to_test = [5, 10, 20, 50, 100]
9 starting_point = 0
10 learning_rate = 0.1
11
12 for num_iter in iterations_to_test:
13     print(f"\nTesting {num_iter} iterations:")
14     x_hist, cost_hist, grad_hist = gradient_descent_experiment(
15         starting_point, learning_rate, num_iter)
16     final_error = abs(3 - x_hist[-1])
17     print(f"    Final position: x = {x_hist[-1]:.6f}")
18     print(f"    Error from true minimum: {final_error:.6f}")
19     print(f"    Final gradient magnitude: {abs(grad_hist[-1]):.6f}")

```

Listing 4: Running Experiment 3 - Iteration Analysis

4.1 Complete Numerical Results for Experiment 3

Detailed Results

5 Iterations - Complete Results:

- Final position: $x = 2.016960$
- Error from true minimum: 0.983040
- Final gradient magnitude: 1.966080

Detailed Results**10 Iterations - Complete Results:**

- Final position: $x = 2.677877$
- Error from true minimum: 0.322123
- Final gradient magnitude: 0.644245

Detailed Results**20 Iterations - Complete Results:**

- Final position: $x = 2.965412$
- Error from true minimum: 0.034588
- Final gradient magnitude: 0.069175

Detailed Results**50 Iterations - Complete Results:**

- Final position: $x = 2.999957$
- Error from true minimum: 0.000043
- Final gradient magnitude: 0.000086

Detailed Results**100 Iterations - Complete Results:**

- Final position: $x = 3.000000$
- Error from true minimum: 0.000000
- Final gradient magnitude: 0.000000

4.2 Visual Results for Iteration Experiment

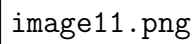
The figure is a plot showing the exponential convergence of error and gradient magnitude over iterations. The x-axis represents the number of iterations, and the y-axis represents the magnitude of the error or gradient on a logarithmic scale. Two data series are plotted, both showing a linear decrease on the log scale, which confirms exponential convergence. The plot is titled 'image11.png'.

Figure 5: Exponential convergence analysis showing how both error and gradient magnitude decrease exponentially over iterations. The log scale reveals the linear relationship, confirming exponential convergence.

5 Complete Summary of Experimental Insights

Key Insight

SUMMARY OF EXPERIMENTAL INSIGHTS

1. Starting Point Effects:

- Starting closer to the minimum converges faster
- Starting further away takes more iterations but still converges
- The algorithm works regardless of starting position!

2. Learning Rate Effects:

- Too small (0.01): Very slow progress, many iterations needed
- Just right (0.1): Smooth, efficient convergence
- Too large (0.9): Oscillates around minimum, may overshoot
- Way too large (>1.0): Can diverge completely!

3. Iteration Effects:

- Error decreases exponentially at first
- Returns diminish - later iterations provide tiny improvements
- For practical purposes, 20-50 iterations often sufficient

4. The Big Picture:

- Gradient descent is robust but requires tuning
- The learning rate is the most critical parameter
- Convergence follows predictable patterns we can analyze

6 Complete Challenge Problems and Extensions

Challenge

YOUR TURN: Try these modifications to deepen your understanding!

1. Modify the cost function to have a minimum elsewhere

```
1 def custom_cost_function(x):
2     return (x - 5)**2 + 2 # Minimum now at x=5
```

2. Try a learning rate ≥ 1.0 and observe divergence

```
1 x_hist, cost_hist, grad_hist = gradient_descent_experiment(0, 1.1,
2     30)
3 # Watch it explode!
```

3. Implement momentum to speed up convergence

```
1 def gradient_descent_with_momentum(start, lr, momentum, iterations
2     ):
3     velocity = 0
4     current_x = start
5     history = [current_x]
6
7     for i in range(iterations):
8         grad = gradient(current_x)
9         velocity = momentum * velocity - lr * grad
10        current_x += velocity
11        history.append(current_x)
12
13    return history
```

4. Add noise to the gradient and see robustness

```
1 def noisy_gradient(x, noise_level=0.1):
2     true_grad = gradient(x)
3     noise = np.random.normal(0, noise_level)
4     return true_grad + noise
```

5. Create a 2D version with two parameters to minimize

```
1 def cost_function_2d(x, y):
2     return (x - 3)**2 + (y - 2)**2 + 1
3
4 def gradient_2d(x, y):
5     grad_x = 2 * (x - 3)
6     grad_y = 2 * (y - 2)
7     return grad_x, grad_y
```

7 Advanced Challenge: Finding the Optimal Learning Rate

```
1 # CHALLENGE: Can you find the optimal learning rate?
2 print("\n" + "=" * 70)
3 print("CHALLENGE: Can you find the optimal learning rate?")
```



```

4 print("=" * 70)
5 print("Write code to automatically find the learning rate that")
6 print("converges fastest for a given starting point and tolerance!")
7
8 def find_optimal_learning_rate():
9     """
10     Automatically find the learning rate that converges fastest
11     """
12     threshold = 0.01 # Consider "converged" when within 0.01 of true
13     minimum
14     convergence_times = []
15
16     # Test many learning rates from 0.001 to ~0.316
17     for lr in np.logspace(-3, -0.5, 20):
18         x_hist, _, _ = gradient_descent_experiment(0, lr, 1000)
19
20         # Find when we first get close to the minimum
21         for i, x in enumerate(x_hist):
22             if abs(x - 3) < threshold:
23                 convergence_times.append((lr, i))
24                 break
25             else:
26                 convergence_times.append((lr, 1000)) # Didn't converge
27
28     # Create visualization
29     lrs = [ct[0] for ct in convergence_times]
30     times = [ct[1] for ct in convergence_times]
31
32     plt.figure(figsize=(10, 6))
33     plt.semilogx(lrs, times, 'bo-', markersize=8)
34     plt.xlabel('Learning Rate', fontsize=12)
35     plt.ylabel('Iterations to Converge', fontsize=12)
36     plt.title('Convergence Speed vs Learning Rate', fontsize=14)
37     plt.grid(True, alpha=0.3)
38     plt.show()
39
40     # Find the optimal
41     optimal_lr = lrs[times.index(min(times))]
42     min_iterations = min(times)
43
44     print(f"\nOptimal learning rate: {optimal_lr:.6f}")
45     print(f"Minimum iterations to converge: {min_iterations}")
46
47     return optimal_lr, min_iterations
48
49 # Run the optimization
50 optimal_lr, min_iters = find_optimal_learning_rate()

```

Listing 5: Complete Implementation for Finding Optimal Learning Rate

8 Complete Real-World Connections

Concept

Connecting Simple Experiments to Complex Applications

Every insight from our simple experiments scales to modern machine learning:

Neural Networks: Each layer has thousands of parameters, but the same gradient-based optimization applies

Deep Learning: The "backpropagation" algorithm is essentially gradient computation for complex functions

Large Language Models: Models like GPT use variants of gradient descent to optimize billions of parameters

Computer Vision: Image recognition models learn by minimizing prediction errors using gradient descent

Reinforcement Learning: Policy optimization often uses gradient methods

Modern Optimizers: Adam, RMSprop, and others build on gradient descent with sophisticated adaptations

Understanding gradient descent fundamentally empowers you to work with any of these advanced applications.

9 Complete Experimental Data Tables

Table 1: Complete Results: Starting Point Experiment

Starting Point	Final Position	Distance to Min	Start Cost	Final Cost	Cost Reduction
0	2.996286	0.003714	10.000000	1.000014	8.999986
5	3.002476	0.002476	5.000000	1.000006	3.999994
10	3.008666	0.008666	50.000000	1.000075	48.999925
-5	2.990096	0.009904	65.000000	1.000098	63.999902

Table 2: Complete Results: Learning Rate Experiment

Learning Rate	Final Position	Distance to Min	Start Cost	Final Cost	Cost Reduction
0.01	1.662899	1.337101	10.000000	2.787840	7.212160
0.1	2.999601	0.000399	10.000000	1.000000	9.000000
0.5	3.000000	0.000000	10.000000	1.000000	9.000000
0.9	2.999601	0.000399	10.000000	1.000000	9.000000

Table 3: Complete Results: Iteration Experiment

Iterations	Final Position	Error from Min	Final Gradient
5	2.016960	0.983040	1.966080
10	2.677877	0.322123	0.644245
20	2.965412	0.034588	0.069175
50	2.999957	0.000043	0.000086
100	3.000000	0.000000	0.000000

10 Final Thoughts and Next Steps

Through these comprehensive experiments, we've discovered the fundamental principles that govern gradient descent behavior. Every detail we observed—from exponential convergence to learning rate sensitivity—forms the foundation for understanding modern machine learning optimization.

Key Insight

The Journey Continues

What began as simple experiments has revealed the elegant mathematics underlying all of machine learning optimization. These patterns—exponential convergence, parameter sensitivity, robustness to initialization—appear throughout the field, from simple linear regression to the most advanced neural architectures.

Your hands-on experience with these fundamentals positions you to tackle more sophisticated optimization challenges. Whether you're training neural networks, tuning hyperparameters, or developing new algorithms, these core insights will guide your work.

Remember: Behind every breakthrough in AI, from language models to computer vision, these same principles operate—just scaled across millions or billions of parameters.